



FACULTAD DE CIENCIAS DE LA VIDA
Y TECNOLOGÍAS

APLICACIONES WEB 2

Apellidos y nombres:

Macias Macias Anthony
Ilaria Michelle Salazar Rezabala
Madelyn Elisa Zambrano Cevallos

Docente:

Ing. John Cevallos

2026-I

Junio 2026

Módulo: Gestión de Pesca

Estudiante: Macias Macias Anthony

Introducción

Durante el desarrollo del proyecto “Pesca Directa Tarqui” se presentaron varios desafíos técnicos relacionados con la arquitectura modular, el manejo de rutas REST y la integración del trabajo en equipo mediante Git y GitHub. Estos problemas permitieron comprender mejor el funcionamiento de Go, la organización de proyectos por capas y la importancia de mantener una estructura consistente entre módulos.

Problema 1: Integración de ramas y conflictos en main.go

Cada integrante desarrolló su módulo en una rama independiente. Al momento de integrar los módulos en la rama principal, se produjeron conflictos en el archivo main.go, ya que todos los integrantes debían registrar sus rutas y almacenes en memoria dentro del mismo punto de entrada de la aplicación.

Como por ejemplo:

```
storePesca := storage.NuevaMemoriaPesca()
```

versus

```
storePedidos := storage.NuevaMemoriaPedidos()
```

```
storeDistribucion := storage.NuevaMemoriaDistribucion()
```

Dificultad

Git no podía determinar automáticamente cuál de las versiones debía conservarse.

Solución

La solución fue mantener las tres inicializaciones y registrar los tres módulos dentro del router principal.

```
storePesca := storage.NuevaMemoriaPesca()
```

```
storePedidos := storage.NuevaMemoriaPedidos()
```

```
storeDistribucion := storage.NuevaMemoriaDistribucion()
```

Posteriormente se registraron todas las rutas dentro de /api/v1.

Aprendizaje

Comprendí que Git no reemplaza la coordinación entre desarrolladores y que una buena estrategia de ramas reduce significativamente los conflictos de integración.

Problema 2: Validación de parámetros en las rutas REST

Ya que:

Los endpoints utilizan rutas parametrizadas como:

```
GET /api/v1/especies/{id} o GET /api/v1/capturas/{id}
```

Al realizar pruebas se detectó que algunos usuarios podían enviar valores inválidos como:

```
GET /api/v1/especies/abc o GET /api/v1/especies/-5
```

Dificultad

El sistema intentaba convertir estos valores a enteros y producía errores.

Solución

Se implementó una función auxiliar llamada parseID().

```
func parseID(r *http.Request) (int, bool)
```

Esta función valida que el parámetro sea un entero positivo antes de continuar con la ejecución del handler.

Si la validación falla se devuelve: 400 Bad Request

Aprendizaje

Comprendí la importancia de validar todos los datos recibidos por el servidor y no confiar en la información enviada por el cliente.

Problema 3: Manejo de IDs inexistentes en operaciones CRUD

Ya que:

Durante las pruebas se realizaron consultas sobre registros inexistentes.

Ejemplo: GET /api/v1/especies/999

Cuando el registro solicitado no existe, el sistema debe responder adecuadamente.

Dificultad

Inicialmente algunas operaciones devolvían respuestas vacías, lo que dificultaba determinar si el registro existía o no.

Solución

Se implementó el patrón:

registro, encontrado := store.BuscarEspeciePorID(id)

Si el valor de encontrado es falso, el sistema responde: 404 Not Found

```
if !encontrado {RespondError(w, http.StatusNotFound, "especie no encontrada")
    return
}
```

Aprendizaje

Aprendí que los códigos HTTP son fundamentales para comunicar correctamente el estado de una operación al cliente.

Reflexión sobre Go

El desarrollo de este proyecto permitió comprender varias características importantes del lenguaje Go.

En primer lugar, Go promueve una arquitectura limpia mediante la separación entre modelos, almacenamiento y handlers. Esto facilita el mantenimiento y escalabilidad del sistema.

También aprendí el uso de interfaces para desacoplar la lógica de negocio de la implementación específica del almacenamiento. Gracias a esto fue posible utilizar una implementación en memoria sin modificar los handlers.

Otro aspecto relevante fue el manejo explícito de errores. A diferencia de otros lenguajes, Go obliga al desarrollador a tratar cada error de forma consciente, lo que mejora la robustez de la aplicación.

Finalmente, el proyecto permitió aplicar conceptos de APIs REST, rutas parametrizadas, serialización JSON, validaciones de entrada y control de versiones con Git, consolidando conocimientos fundamentales para el desarrollo backend moderno.

Módulo: Gestión de Pedidos

Estudiante: Ilaria Michelle Salazar Rezabala

Los 3 problemas más difíciles que enfrenté

1. El servidor me respondía 400

Al hacer un POST a /clientes me devolvía JSON inválido: EOF y no sabía qué estaba mal. Después de revisar me di cuenta de que estaba enviando la petición en Postman sin body, sin seleccionar raw ni JSON. Aprendí que el handler necesita recibir el body correctamente decodificado y que, si no llega nada, Go lo toma como un error de lectura.

2. El usuario_id se pisaba con 0 al hacer un PUT

Cuando actualizaba un cliente sin mandar todos los campos, los que no enviaba se sobrescribían con su valor por defecto, en el caso de los enteros ese valor es 0. Me pasó con usuario_id que después de un PUT aparecía como 0 aunque antes tenía valor. Lo resolví agregando verificaciones en el storage para que, si el campo viene vacío o en cero, conserve el valor que ya tenía guardado.

3. Mutex

Al principio agregué el sync.Mutex porque lo vi en el código de referencia, pero no entendía bien por qué estaba ahí. Con el tiempo entendí que el servidor puede recibir varias peticiones al mismo tiempo y sin el mutex dos peticiones podían modificar los mismos datos a la vez y corromperlos. Una vez que lo entendí quedó claro que el Lock y el defer Unlock van en cada función que toca los datos.

Reflexión personal

Go es un lenguaje que al principio se siente extraño, pero con las prácticas de clase y este proyecto fui entendiéndolo mejor poco a poco. Lo que más me llamó la atención es que el código no es tan extenso como en otros lenguajes que he usado, porque la forma en que Go estructura las cosas permite hacer más con menos líneas. Por ejemplo, manejar rutas, respuestas JSON y validaciones sin necesitar librerías enormes ni configuraciones complicadas. Siento que las prácticas que fuimos haciendo en clase me dieron la base para poder aplicarlo en este proyecto, y aunque todavía me falta mucho por aprender, ya no se me hace tan difícil leer y escribir código en Go como al principio.

Módulo: Rutas de Distribución

Estudiante: Madelyn Elisa Zambrano Cevallos

Problemas a enfrentar

1. Problema: Constructores no definidos

En mi archivo routes-rutas.go aparecía el error:

undefined: NewPuntoHandler

undefined: NewTransportistaHandler

undefined: NewEntregaHandler

Explicación: Eso pasaba porque las funciones constructoras (NewXHandler) no estaban implementadas. En Go, cuando se define un struct como PuntoHandler, necesita un constructor que inicialice la estructura con su store.

Solución aplicada:

```
func NewPuntoHandler(s storage.PuntoStore) *PuntoHandler {
    &PuntoHandler{store: s}
}
```

Reflexión: Este error es común cuando se replica el patrón de un handler pero se olvida crear el constructor. La ventaja de Go es que la compilación estricta detecta inmediatamente la ausencia de funciones.

2. Problema: parseID devolviendo tipos inconsistentes

El compilador mostraba:

invalid operation: err != nil (mismatched types bool and untyped nil)

Explicación: El parseID estaba definido para devolver (int, bool), pero en los handlers se usaba como (int, error). Esto provocaba que err fuera tratado como un bool.

Solución aplicada: Se redefinió parseID para devolver (uint, bool) y se cambió el uso en los handlers:

```
id, ok := parseID(r)
if !ok {
    RespondError(w, http.StatusBadRequest, "ID inválido")
    return
}
```

Reflexión: Go obliga a ser explícito con los tipos. Esto evita ambigüedades y asegura que el código sea más claro y seguro, aunque al inicio pueda parecer rígido.

3. Problema: Mismatch entre int y uint

En ObtenerUno aparecía:

cannot use id (variable of type int) as uint value in argument to
h.store.ObtenerRutaPorID

Explicación: El store esperaba un uint, pero parseID devolvía int. Esto generaba incompatibilidad.

Solución aplicada:

```
id, err := strconv.ParseUint(idStr, 10, 64)
if err != nil || id == 0 {
    return 0, false
}
return uint(id), true
```

Reflexión: Este tipo de errores muestra cómo Go fuerza la coherencia en todo el flujo de datos. Aunque puede ser tedioso, garantiza que no haya conversiones implícitas peligrosas.

Conclusión

En conclusión, trabajar con Go en handlers de rutas de distribución me enseñó que:

- La compilación estricta es tu mejor aliada: detecta errores de tipos y funciones faltantes antes de ejecutar.
 - El manejo explícito de errores (if err != nil) obliga a pensar en cada caso: campos obligatorios, IDs inexistentes, duplicados.
 - La simplicidad del lenguaje (sin herencia compleja, sin sobrecarga) hace que el código sea más fácil de leer y mantener, aunque al inicio parezca repetitivo.
- Todo esto quiere decir que los problemas que surgieron (constructores faltantes, tipos inconsistentes, conversiones entre int y uint) son típicos de escribir código a mano, pero también son oportunidades para demostrar dominio del lenguaje y capacidad de depuración.

Enlace a GitHub:

[madeluki2/Proyecto_Aplicaciones_Web_II at feature/rutas-de-distribucion](https://github.com/madeluki2/Proyecto_Aplicaciones_Web_II/tree/main/feature/rutas-de-distribucion)

Enlace a los videos:

[Videos AW2](#)